

Performance Optimization

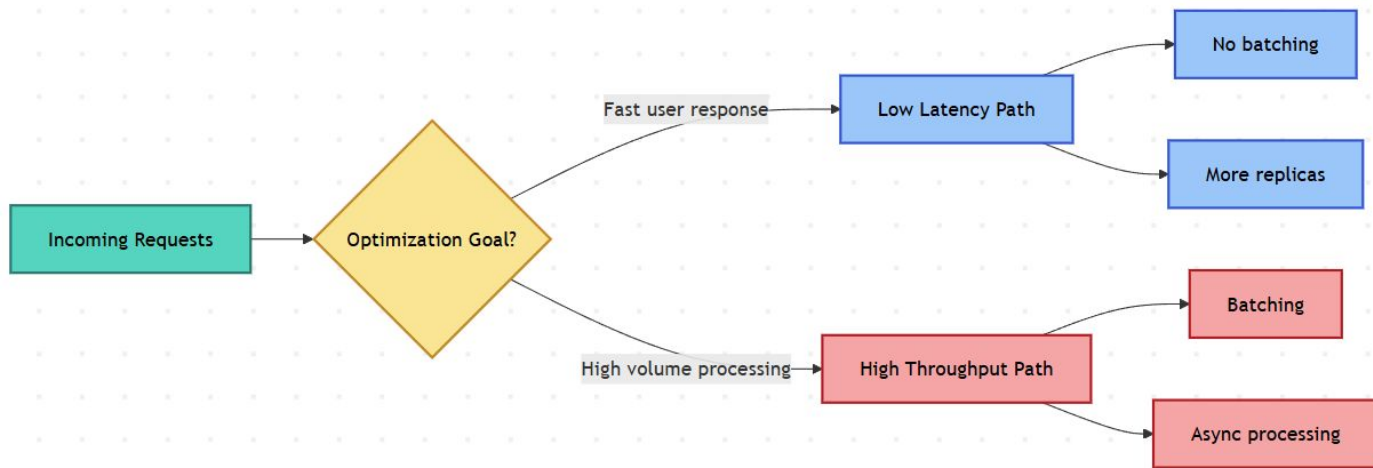
System Design Crash Course: Quick Revision Before Interviews

Rahul Rajat Singh



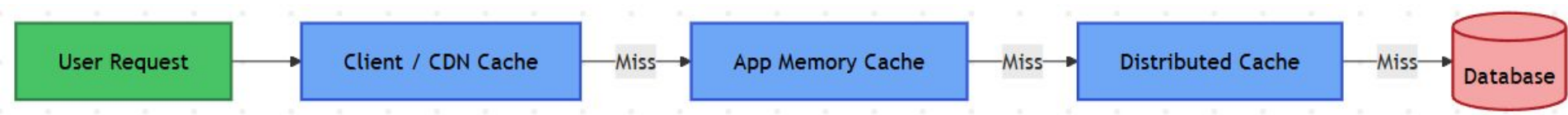
Latency vs Throughput

- Latency = time per request
- Throughput = requests per time
- Optimize for user vs system
- Often inversely related
- Depends on workload goals



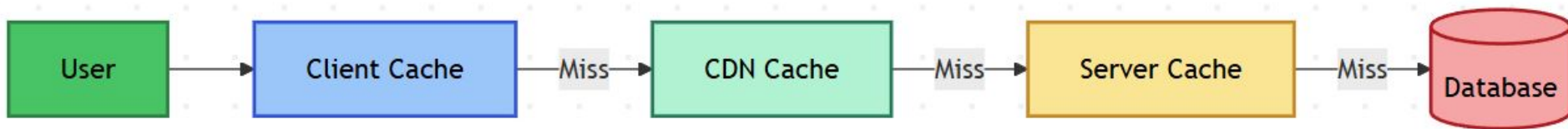
Caching Fundamentals

- Store data closer to compute
- Reduce latency and load
- Multiple cache layers exist
- Read-heavy workloads benefit most
- Consistency vs performance trade-off



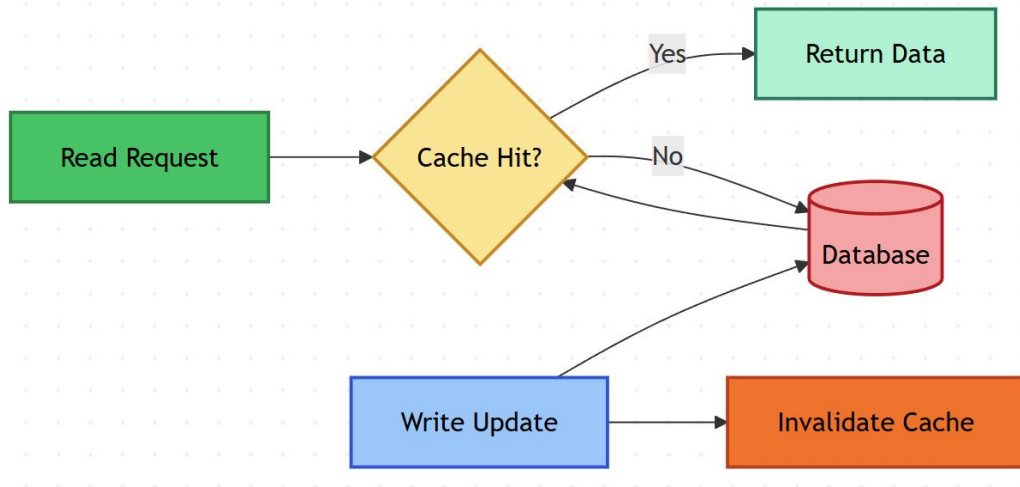
Cache Placement

- Client-side cache: closest to user
- CDN cache: edge-level acceleration
- Server-side cache: shared backend
- Closer cache reduces latency
- Placement affects consistency, scale



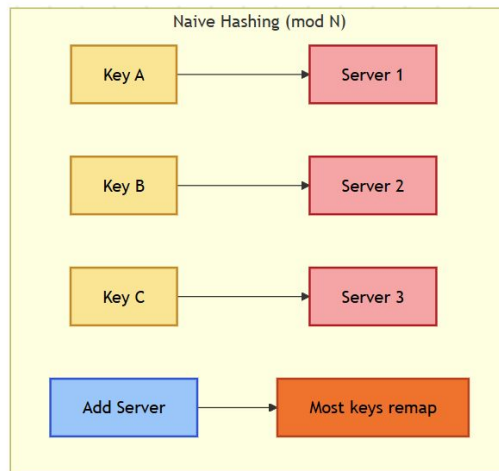
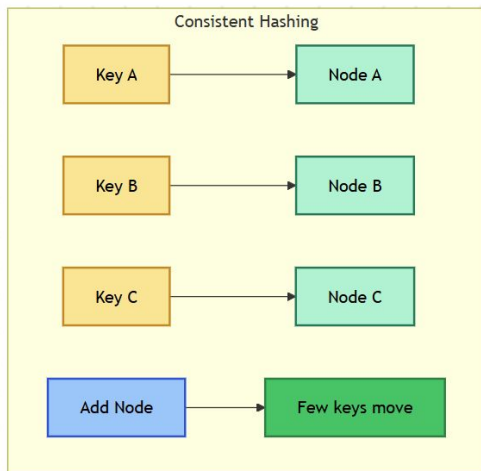
Cache Invalidation

- Remove or update stale data
- Hardest caching problem
- TTL, write-through, explicit delete
- Consistency vs performance trade-off
- Wrong invalidation breaks correctness



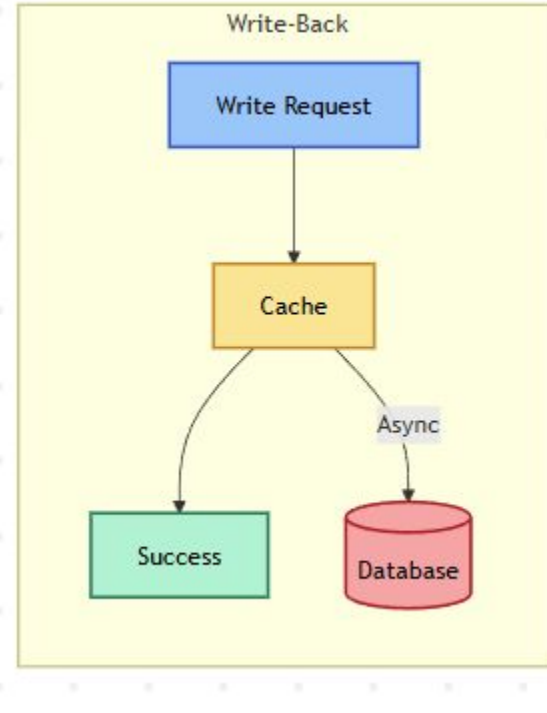
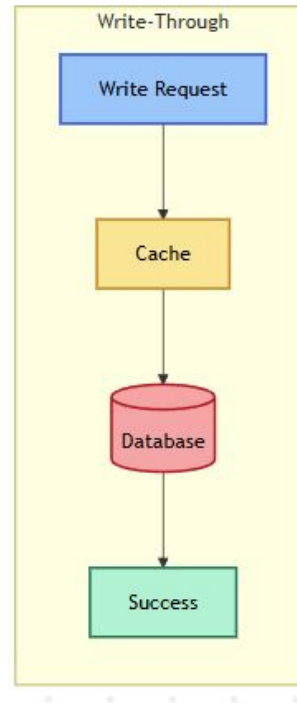
Consistent Hashing

- Distributes keys on hash ring
- Minimizes key movement
- Nodes added without reshuffle
- Used in caches and sharding
- Slight imbalance, higher complexity



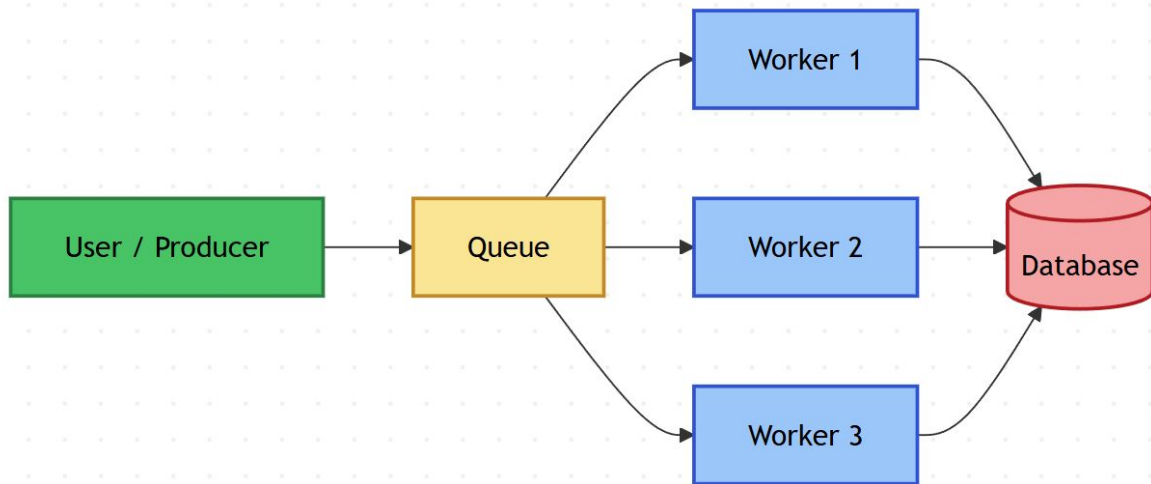
Write-Through vs Write-Back Cache

- Write-through: cache and database
- Write-back: cache first, async DB
- Consistency vs write latency
- Data safety vs performance
- Failure handling differs



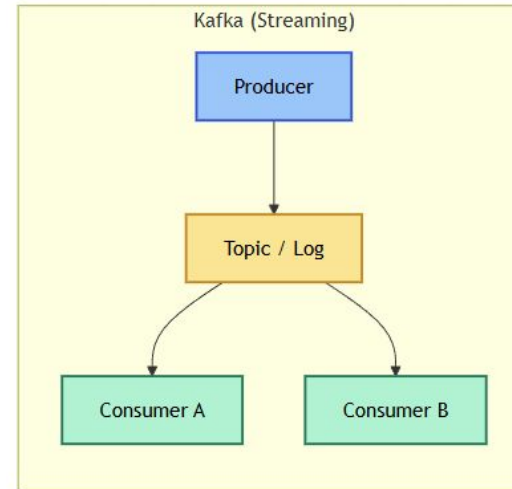
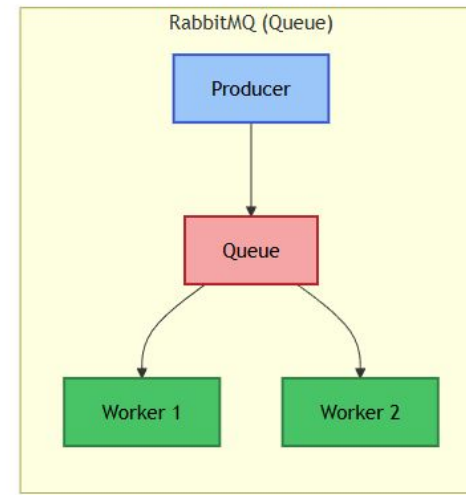
Queues and Async Processing

- Decouple producers and consumers
- Buffer traffic spikes safely
- Async improves responsiveness
- Enables independent scaling
- Adds eventual consistency



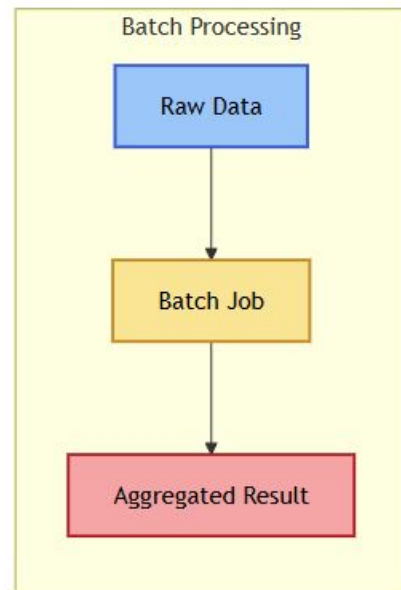
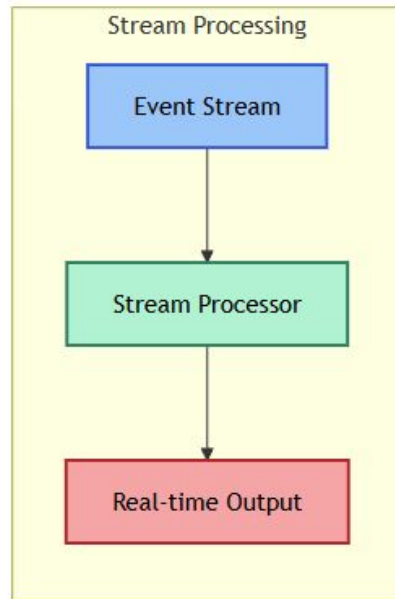
Kafka vs RabbitMQ

- Kafka: distributed event streaming
- RabbitMQ: message queuing broker
- Throughput vs routing flexibility
- Pull-based vs push-based consumption
- Replay support vs transient messages



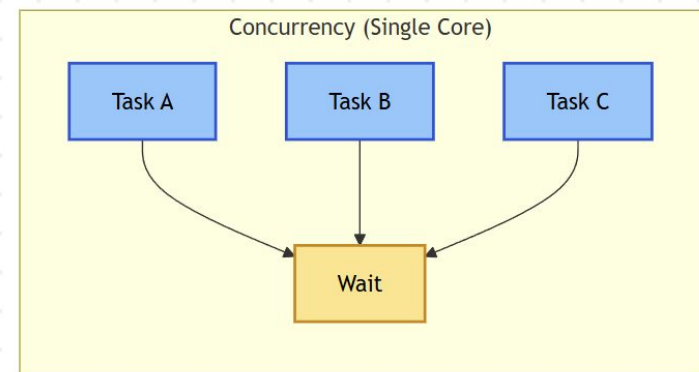
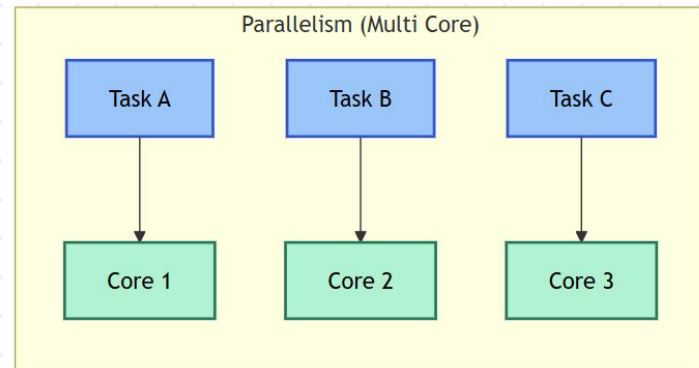
Batch vs Stream Processing

- Batch: process data in chunks
- Stream: process events continuously
- Latency vs efficiency trade-off
- Cost optimized vs freshness optimized
- Complexity differs significantly



Concurrency vs Parallelism

- Concurrency: managing multiple tasks
- Parallelism: executing tasks simultaneously
- I/O-bound vs CPU-bound workloads
- Single core vs multiple cores
- Often confused, not identical



Thread Pools

- Fixed set of worker threads
- Tasks queued for execution
- Bounds resource usage
- Prevents thread explosion
- Latency vs throughput trade-off

